



UNIVERSIDAD NACIONAL DEL CENTRO DEL PERU
FACULTAD DE INGENIERIA DE SISTEMAS
DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS
PROGRAMA DE INGENIERÍA DE SISTEMAS



Guía Práctica SEMANA 06

Asignatura: Desarrollo de Aplicaciones Web (IS093A)

Unidad I: Desarrollo Web Frontend

Tema: React Hooks: useState, useEffect, useContext, useRef, useReducer, useCallback, useMemo y Hooks Personalizados

Modalidad: Presencial / Laboratorio

APELLIDOS Y NOMBRES: Barja Ortiz Erick Gerson

OBJETIVO DE LA PRÁCTICA

Dominar el ciclo de renderizado de React mediante el uso estratégico de Hooks nativos, aplicar useEffect para efectos secundarios y limpieza, gestionar estado global con useContext, optimizar rendimiento con useMemo/useCallback, manejar referencias inmutables con useRef, refactorizar lógica compleja con useReducer, y encapsular comportamientos reutilizables en Hooks Personalizados. Validar el flujo de re-renders con React DevTools Profiler.

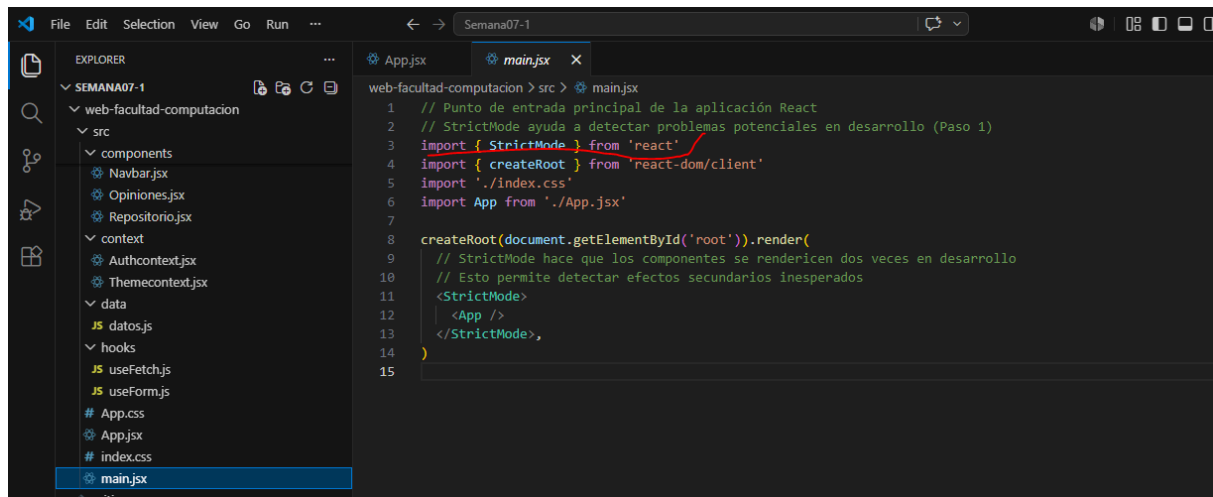
RECURSOS REQUERIDOS

- Node.js LTS (≥20.x) + npm/pnpm
- Visual Studio Code (Extensiones: ES7+ React Snippets, React Developer Tools, ESLint, Error Lens)
- Navegador moderno + DevTools (Network, Performance, Console)
- Proyecto base Vite + React (heredado de Semana 06 o scaffold nuevo)
- API de prueba: <https://jsonplaceholder.typicode.com/users> o <https://dummyjson.com/products>
- Documentación oficial: react.dev/reference/react
- Conexión a internet

EJERCICIO PASO A PASO

1. Revisar ciclo de render: identificar qué dispara rerender en useState vs props.

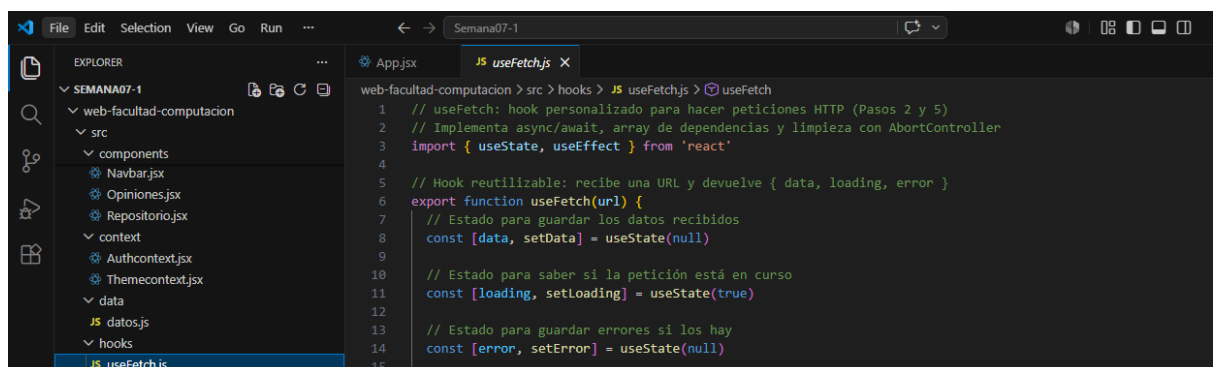
Configurar React.StrictMode y Profiler



The screenshot shows the VS Code editor with the file explorer on the left and the main.jsx file open in the editor. The file explorer shows the project structure for 'SEMANA07-1' with folders 'web-facultad-computacion', 'src', 'components', 'context', 'data', 'hooks', and 'App.css'. The main.jsx file is selected in the file explorer and open in the editor. The code in main.jsx is as follows:

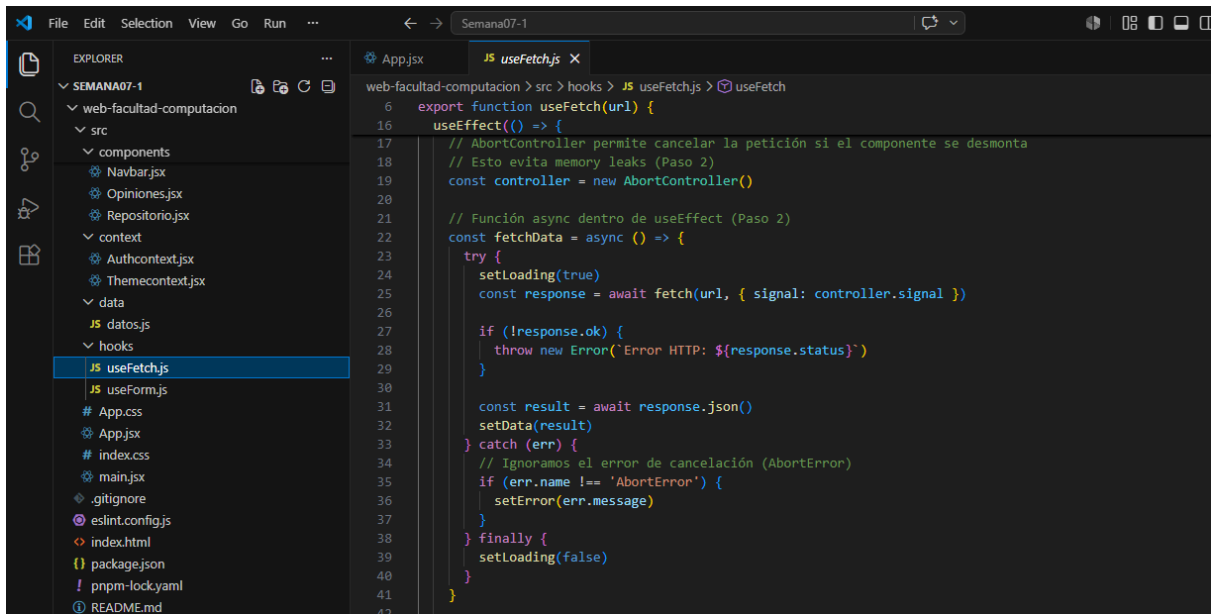
```
1 // Punto de entrada principal de la aplicación React
2 // StrictMode ayuda a detectar problemas potenciales en desarrollo (Paso 1)
3 import { StrictMode } from 'react'
4 import { createRoot } from 'react-dom/client'
5 import './index.css'
6 import App from './App.jsx'
7
8 createRoot(document.getElementById('root')).render(
9   // StrictMode hace que los componentes se rendericen dos veces en desarrollo
10  // Esto permite detectar efectos secundarios inesperados
11  <StrictMode>
12    <App />
13  </StrictMode>,
14 )
15
```

2. Refactorizar fetch de API: migrar .then() a async/await dentro de useEffect, añadir array de dependencias y función de limpieza (AbortController)

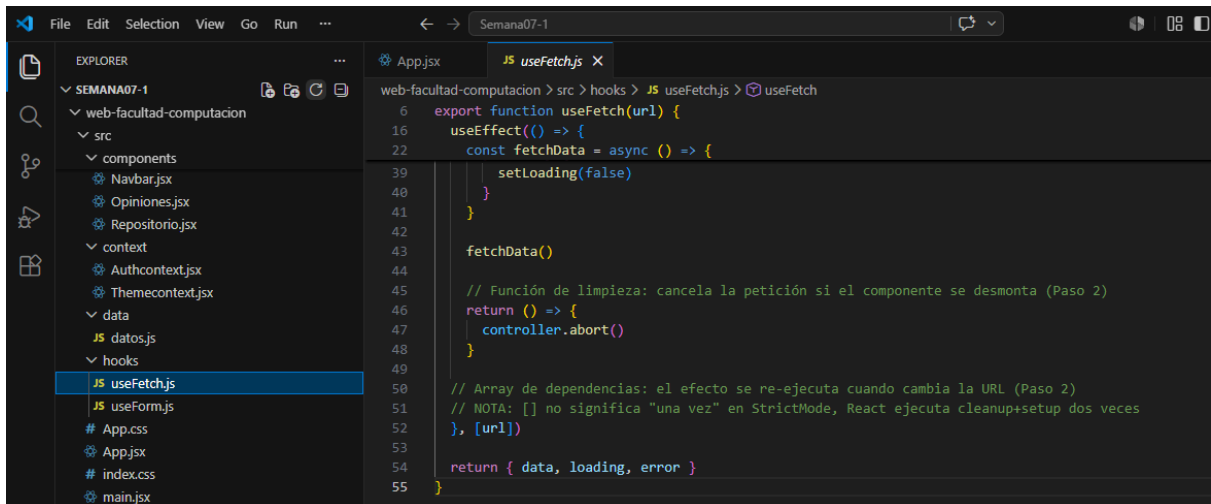


The screenshot shows the VS Code editor with the file explorer on the left and the useFetch.js file open in the editor. The file explorer shows the project structure for 'SEMANA07-1' with folders 'web-facultad-computacion', 'src', 'components', 'context', 'data', 'hooks', and 'App.css'. The useFetch.js file is selected in the file explorer and open in the editor. The code in useFetch.js is as follows:

```
1 // useFetch: hook personalizado para hacer peticiones HTTP (Pasos 2 y 5)
2 // Implementa async/await, array de dependencias y limpieza con AbortController
3 import { useState, useEffect } from 'react'
4
5 // Hook reutilizable: recibe una URL y devuelve { data, loading, error }
6 export function useFetch(url) {
7   // Estado para guardar los datos recibidos
8   const [data, setData] = useState(null)
9
10  // Estado para saber si la petición está en curso
11  const [loading, setLoading] = useState(true)
12
13  // Estado para guardar errores si los hay
14  const [error, setError] = useState(null)
15
```

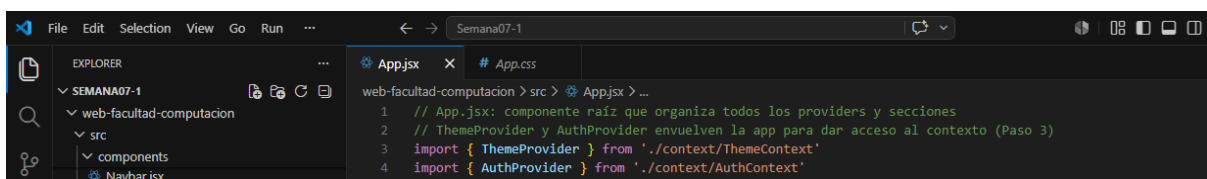
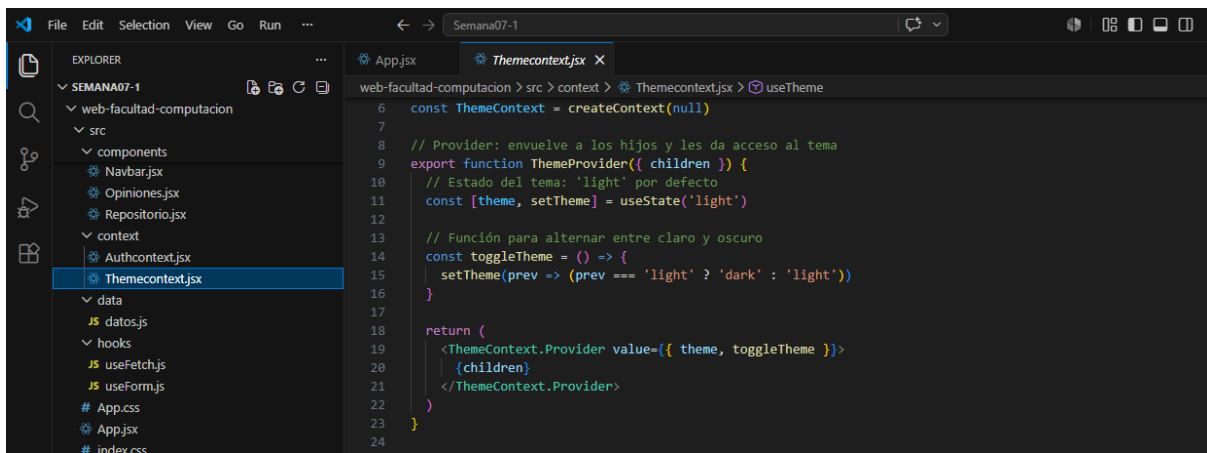
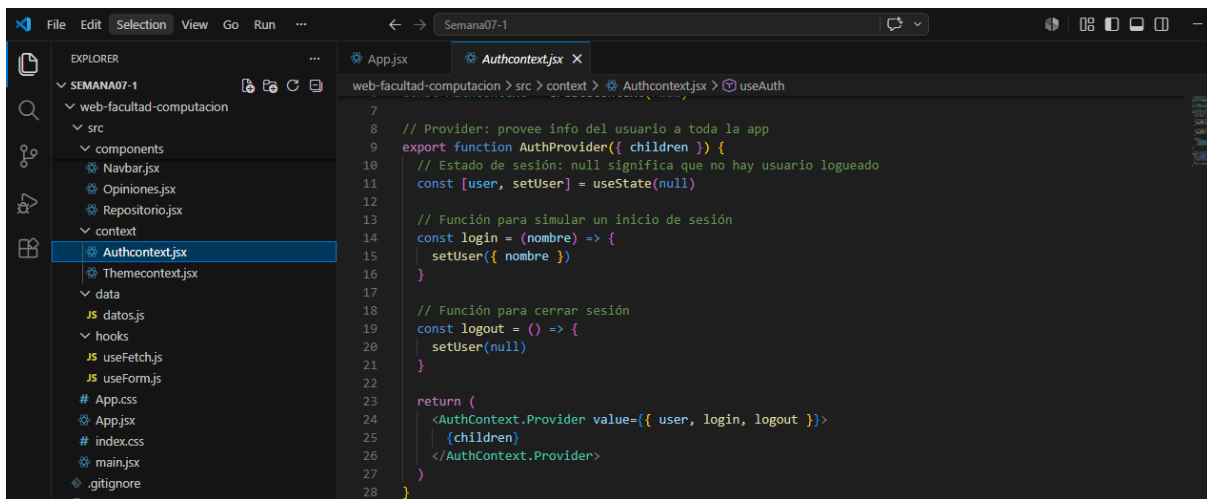
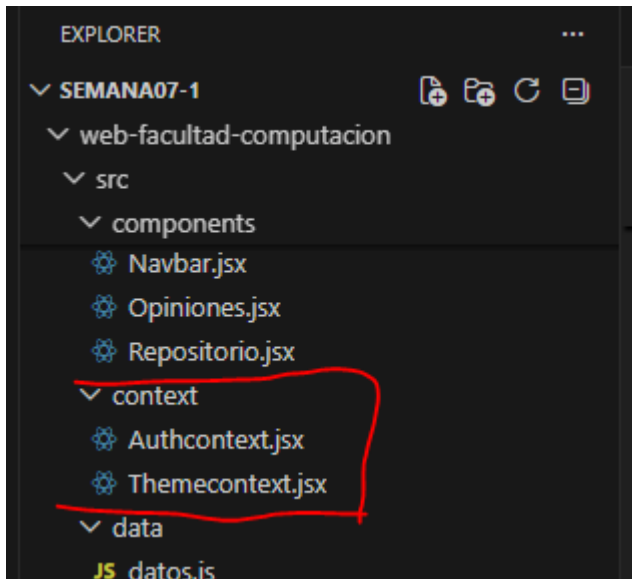


```
web-facultad-computacion > src > hooks > JS useFetchjs > useFetch
6 export function useFetch(url) {
16   useEffect(() => {
17     // AbortController permite cancelar la petición si el componente se desmonta
18     // Esto evita memory leaks (Paso 2)
19     const controller = new AbortController()
20
21     // Función async dentro de useEffect (Paso 2)
22     const fetchData = async () => {
23       try {
24         setLoading(true)
25         const response = await fetch(url, { signal: controller.signal })
26
27         if (!response.ok) {
28           throw new Error(`Error HTTP: ${response.status}`)
29         }
30
31         const result = await response.json()
32         setData(result)
33       } catch (err) {
34         // Ignoramos el error de cancelación (AbortError)
35         if (err.name !== 'AbortError') {
36           setError(err.message)
37         }
38       } finally {
39         setLoading(false)
40       }
41     }
42   })
}
```

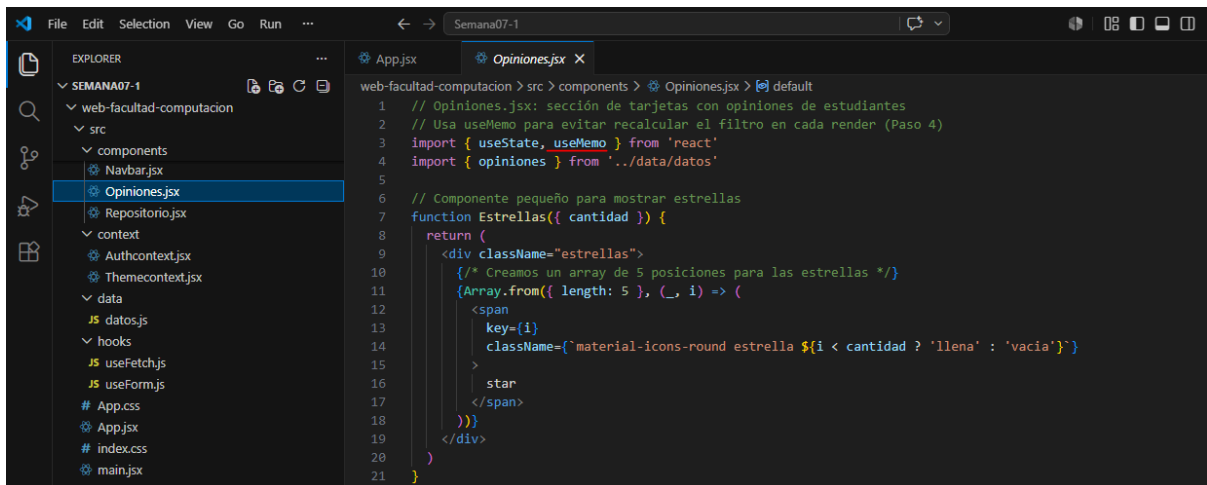


```
web-facultad-computacion > src > hooks > JS useFetchjs > useFetch
6 export function useFetch(url) {
16   useEffect(() => {
22     const fetchData = async () => {
39       setLoading(false)
40     }
41   }, [url])
42
43   fetchData()
44
45   // Función de limpieza: cancela la petición si el componente se desmonta (Paso 2)
46   return () => {
47     controller.abort()
48   }
49
50   // Array de dependencias: el efecto se re-ejecuta cuando cambia la URL (Paso 2)
51   // NOTA: [] no significa "una vez" en StrictMode, React ejecuta cleanup+setup dos veces
52   //, [url])
53
54   return { data, loading, error }
55 }
```

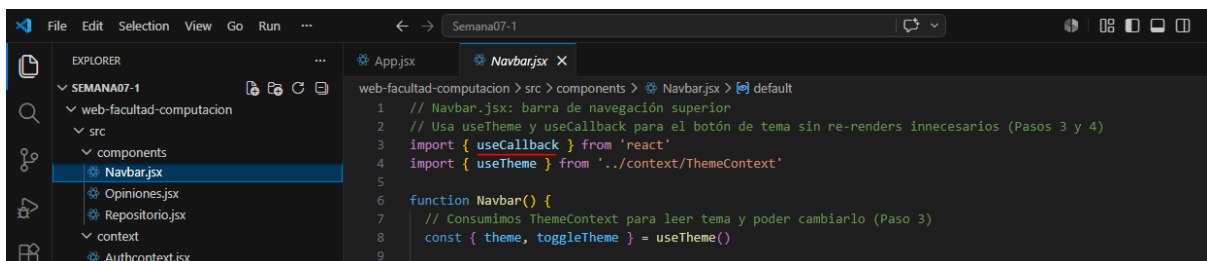
3. Implementar useContext para tema global o estado de sesión. Crear Provider y consumirlo sin prop drilling



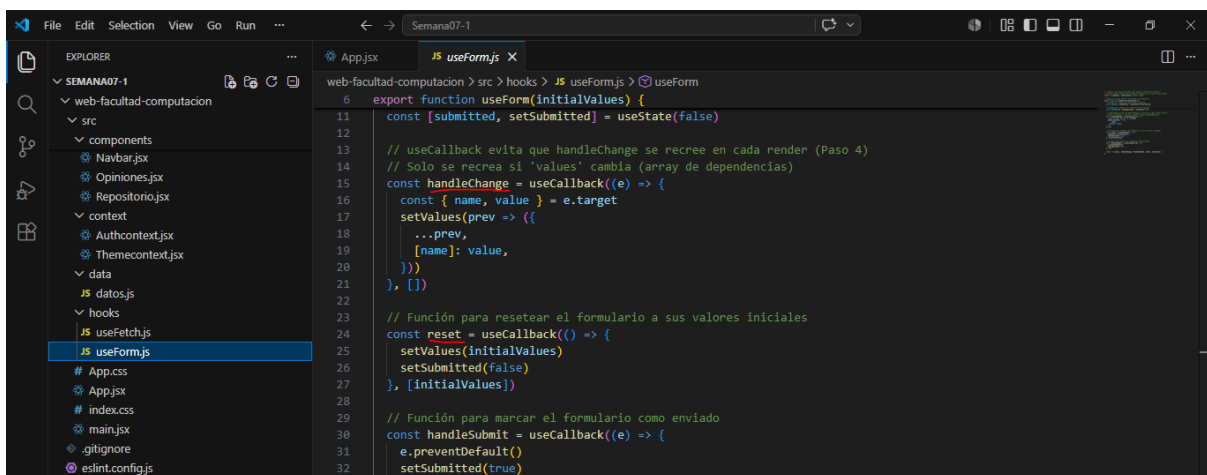
4. Optimizar render: envolver filtros pesados en useMemo, estabilizar callbacks en useCallback, usar useRef para DOM/input focus sin rerender



```
1 // Opiones.jsx: sección de tarjetas con opiniones de estudiantes
2 // Usa useMemo para evitar recalcular el filtro en cada render (Paso 4)
3 import { useState, useMemo } from 'react'
4 import { opiniones } from '../data/datos'
5
6 // Componente pequeño para mostrar estrellas
7 function Estrellas({ cantidad }) {
8   return (
9     <div className="estrellas">
10       /* Creamos un array de 5 posiciones para las estrellas */
11       {Array.from({ length: 5 }, (_, i) => (
12         <span
13           key={i}
14           className={material-icons-round estrella ${i < cantidad ? 'llena' : 'vacía'}}
15         >
16           star
17         </span>
18       ))}
19     </div>
20   )
21 }
```



```
1 // NavBar.jsx: barra de navegación superior
2 // Usa useTheme y useCallback para el botón de tema sin re-renders innecesarios (Pasos 3 y 4)
3 import { useCallback } from 'react'
4 import { useTheme } from '../context/ThemeContext'
5
6 function NavBar() {
7   // Consumimos ThemeContext para leer tema y poder cambiarlo (Paso 3)
8   const { theme, toggleTheme } = useTheme()
9 }
```

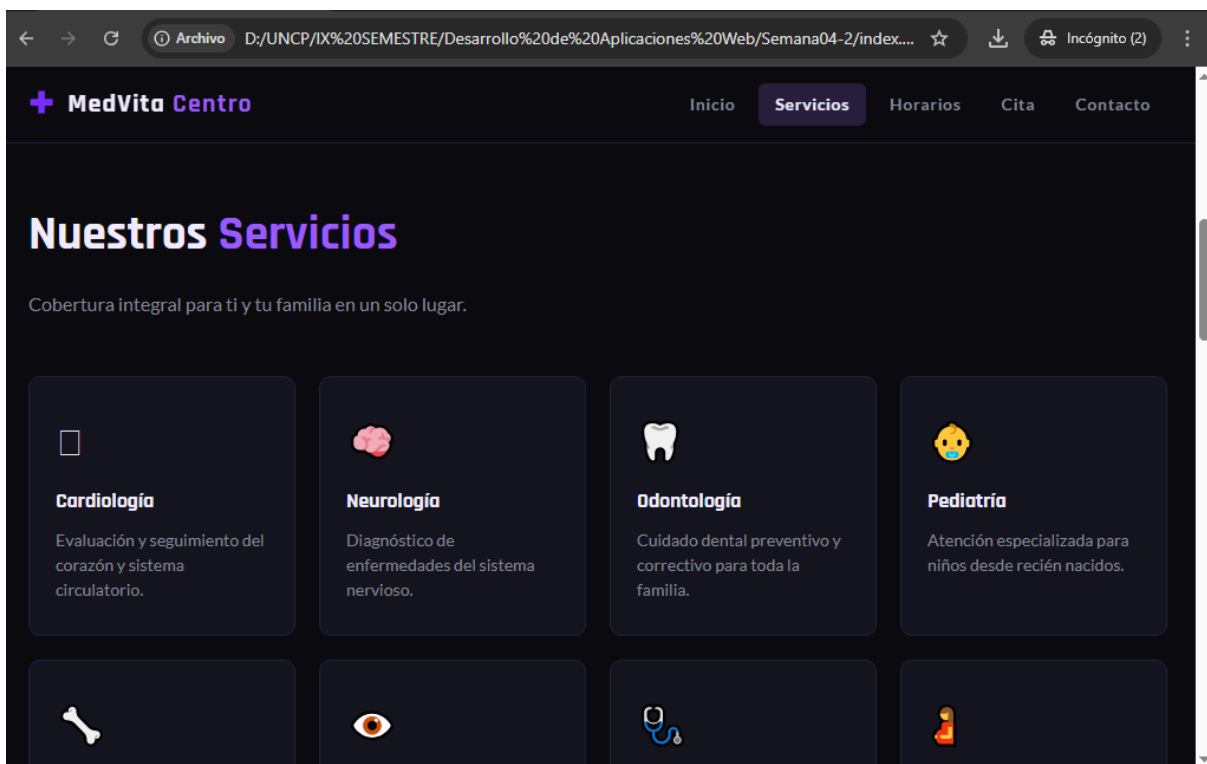


```
1 // useForm.js
2 // useForm: Hook personalizado para manejar formularios
3 // useForm: Hook personalizado para manejar formularios
4 // useForm: Hook personalizado para manejar formularios
5 // useForm: Hook personalizado para manejar formularios
6 export function useForm(initialValues) {
7   const [submitted, setSubmitted] = useState(false)
8
9   // useCallback evita que handleChange se cree en cada render (Paso 4)
10   // Solo se recrea si 'values' cambia (array de dependencias)
11   const handleChange = useCallback((e) => {
12     const { name, value } = e.target
13     setValues(prev => ({
14       ...prev,
15       [name]: value,
16     })), [value]
17   })
18
19   // Función para resetear el formulario a sus valores iniciales
20   const reset = useCallback(() => {
21     setValues(initialValues)
22     setSubmitted(false)
23   }, [initialValues])
24
25   // Función para marcar el formulario como enviado
26   const handleSubmit = useCallback((e) => {
27     e.preventDefault()
28     setSubmitted(true)
29   })
30 }
```

5. Extraer lógica repetitiva en Hook Personalizado (useFetch o useForm). Validar reglas de Hooks y testear reutilización

```
web-facultad-computacion > src > hooks > JS useFetchjs > useFetch
6 export function useFetch(url) {
16   useEffect(() => {
18     // Esto evita memory leaks (Paso 2)
19     const controller = new AbortController()
20
21     // Función async dentro de useEffect (Paso 2)
22     const fetchData = async () => {
23       try {
24         setLoading(true)
25         const response = await fetch(url, { signal: controller.signal })
26
27         if (!response.ok) {
28           throw new Error(`Error HTTP: ${response.status}`)
29         }
30
31         const result = await response.json()
32         setData(result)
33       } catch (err) {
34         // Ignoramos el error de cancelación (AbortError)
35         if (err.name !== 'AbortError') {
36           setError(err.message)
37         }
38       } finally {
39         setLoading(false)
40       }
41     }
42   })
43   fetchData()
```

6. PAGINA WEB IMPLEMENTADA LOCALMENTE



←

→

↺

📄 Archivo

D:/UNCP/IX%20SEMESTRE/Desarrollo%20de%20Aplicaciones%20Web/Semana04-2/index....

☆

⬇️

🔒 Incógnito (2)

⋮

+

MedVita Centro

Inicio

Servicios

Horarios

Cita

Contacto

Horarios de Atención

Día	Consulta General	Especialidades	Urgencias
Lunes - Viernes	7:00 - 20:00	8:00 - 18:00	24 hrs.
Sábado	8:00 - 14:00	9:00 - 13:00	24 hrs.
Domingo	Cerrado	Cerrado	24 h



Urgencias disponibles las 24 horas

Llama directo: +51 999 888 777

Reserva tu Cita aqui

←

→

↺

📄 Archivo

D:/UNCP/IX%20SEMESTRE/Desarrollo%20de%20Aplicaciones%20Web/Semana04-2/i...

🔍

☆

⬇️

🔒 Incógnito (2)

⋮

+

MedVita Centro

Inicio

Servicios

Horarios

Cita

Contacto

Reserva tu Cita aqui

Completa el formulario y nos pondremos en contacto

NOMBRE COMPLETO

Ej. Luis Hernández López

CORREO ELECTRÓNICO

ejemplo@gmail.com

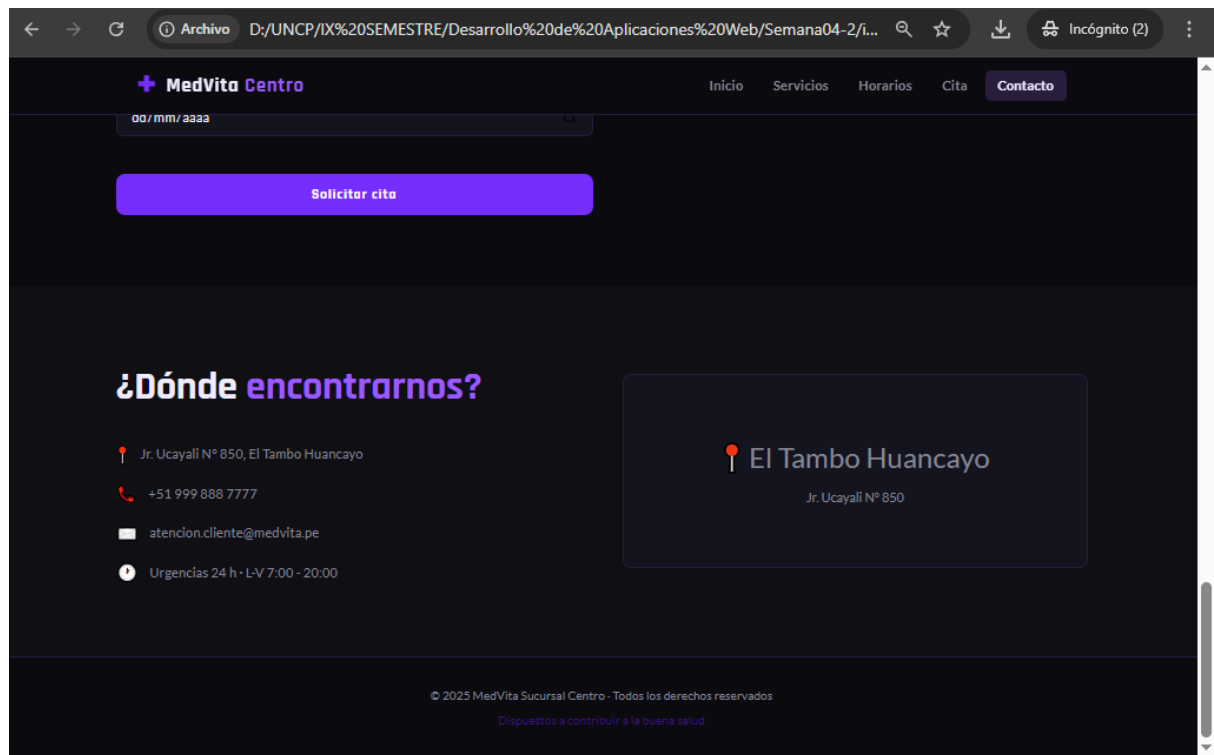
ESPECIALIDAD

— Seleccionar una Esp. —

FECHA PREFERIDA

dd/mm/aaaa

Solicitar cita



Link en GItHub: <https://github.com/GErickOBTS/Semana07-1>